

基于 xv6-RISC-V 的 静态优先级调度与 Aging 防饥饿机制

操作系统设计综合实践课程汇报

天津大学软件学院 · 第 35 组

先明确问题，再分工实现

I 问题与目标

xv6 原有轮转调度把可运行进程近似等价对待，难以表达任务之间的紧急程度。

- 用静态优先级表达调度倾向；
- 保留同优先级进程的基本公平；
- 用 Aging 降低低优先级进程长期等待的风险。

I 统一约定

0 最高

10 默认

31 最低

I 五人分工

赖弈铭

Aging、等待时间累计与防饥饿逻辑

刘文睿

静态优先级调度与同级公平

吴雨桐

测试设计、集成与回归验证

党正清

优先级系统调用与用户态 `nice`

袁晟棋

进程字段、默认值、`fork` 继承与集成

实现边界：保持 xv6 原有状态转换与锁语义，不引入 MLFQ、CFS 或复杂队列。

从用户命令

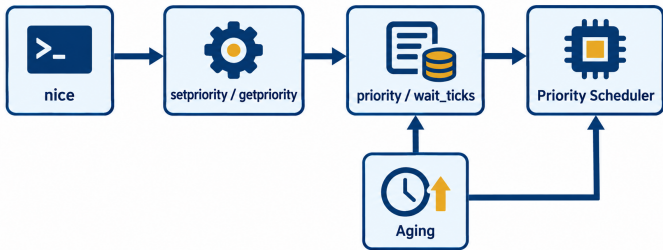


图 1 优先级控制路径：用户态命令进入系统调用，最终影响进程字段与调度选择。

优先级语义

数字越小，优先级越高；合法范围为 0-31，默认值为 10。

关键状态

`priority` 描述当前静态优先级，`wait_ticks` 为 Aging 提供等待依据。

调度入口

`scheduler` 只选择 `RUNNABLE` 进程，并在真正切换前再次确认状态。

进程优先级数据结构

字段设计

<code>priority</code>	静态优先级，合法范围 0–31。
<code>wait_ticks</code>	进程处于 <code>RUNNABLE</code> 状态时的等待累计。

统一语义

0 最高

10 默认

31 最低

生命周期处理

创建进程	设置默认优先级，清零 <code>wait_ticks</code> 。
<code>fork</code> 子进程	继承父进程优先级，等待计数重新置零。
调度成功	被选中的进程清零等待计数。
释放并复用	重置字段，避免旧状态影响新进程。

锁保护

`fork` 读取父进程优先级前先取得父进程锁，复制受保护的优先级快照。

静态优先级调度

两轮扫描

第 1 轮	扫描进程表；对等待进程执行 Aging，先确定需要服务的优先级层。并找出最小合法优先级。
第 2 轮	从 <code>next_index</code> 开始循环，选择该优先级下的第一个 <code>RUNNABLE</code> 进程。再决定这一轮由谁运行。
切换前	重新检查状态并获得进程锁，切换后推进 <code>next_index</code> 。适配多 CPU，减少状态竞争和同级偏爱。

核心判断

$$p_{\text{best}} = \min\{p.\text{priority} \mid p.\text{state} = \text{RUNNABLE}\}$$

数值越小，进程越早获得调度机会。

公平性与 Aging

`next_index` 只在同优先级候选者之间轮转；等待达到 `AGING_INTERVAL=50` 后，优先级向 0 方向提升一级。

系统调用与用户态命令

接口链路

<code>setpriority</code>	设置目标 PID 的优先级。
<code>getpriority</code>	查询目标 PID 的优先级。
系统调用号	23 与 24, 沿用 xv6 的注册流程。
用户态入口	在 <code>user.h</code> 、 <code>usys.pl</code> 中保持一致。

内核实现

`sysproc.c` 负责取参数, `proc.c` 在遍历进程表时逐项加锁, 成功后释放锁再返回。

参数检查

- PID 必须是正数, 且目标进程存在;
- 优先级必须满足 $0 \leq p \leq 31$;
- 手动设置后清零 `wait_ticks`, 避免旧等待历史立即触发 Aging。

命令形式

`nice pid` 查询优先级; `nice pid p` 设置优先级并打印结果。

调度统计

单核混合负载：完成历时中位数 (uptime ticks, n=3)

优先级	轮转 RR	静态优先级	加入 Aging
3 (高)	9	4	3
10 (默认)	9	8	6
20 (低)	9	12	10

三个进程同时释放，固定工作量，重复 3 次；数值为从释放到计算完成的中位 tick。

指标如何读取

METRIC 记录开始延迟和完成历时；wait_ticks 是调度扫描累计值，不能当作实际运行时间。

结果怎么理解

优先级 3 的完成历时由 RR 的 9 ticks 降至 4；优先级 20 延后完成，体现服务次序变化，不代表总吞吐量提高。

用户态测试

重复回归结果

单 CPU

专项测试各重复 3 次，快速 `usertests -q` 通过 (7 PASS)。

3 CPU

专项测试各重复 3 次，完整 `usertests` 通过 (7 PASS)。

策略对照

RR、priority、Aging 各重复 3 次；12 PASS、4 EXPECTED_FAIL、0 FAIL。

基础覆盖

`prioritytest` 检查 API 边界、`nice`、`fork` 继承、同级公平及不同优先级；`agingtest` 检查持续竞争下的低优先级进程。

负向对照也符合预期

关闭 Aging 后，3/3 轮在 256 ticks 超时，目标仍为优先级 3，8 个竞争进程存活；启用 Aging 后 3/3 轮通过，目标降至优先级 0。

证据与版本

原始 QEMU 日志、构建日志、CSV 和 JSON 已归档；这些结果对应测试报告记录的源码提交，不是对当前 `main` 的重新回归。

归档来自队友的 `docs/task5-test-report` 分支；single / SMP 为 7c22218，compare 为 a2e5630。

谢 谢 大 家

第 35 组 · 天津大学软件学院